



POLITECNICO
MILANO 1863

List It: Design Document



Authors: Leonardo Ratti, Mattia Peruzzi

Professor: Luciano Baresi

Course: Design and Implementation of Mobile Applications

Academic Year: 2025-2026

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Requirements	1
1.2.1	Functional Requirements	1
1.2.2	Non-Functional Requirements	3
1.3	Features Implemented	3
1.3.1	Onboarding and Multi-Mode Access	3
1.3.2	Credential Recovery and Email Verification	3
1.3.3	User Profile and Security	3
1.3.4	Shopping List Management	3
1.3.5	Product Entry with Smart Categorization	3
1.3.6	Supermarket and Category Management	3
1.3.7	Supermarket Adaptation	3
1.3.8	Shopping List Initialization	3
1.3.9	AI-Assisted Recipe Generation	4
1.3.10	Background Recipe Notifications	4
1.3.11	Purchase Registration and History	4
1.3.12	Receipt Scanning with OCR and AI	4
1.3.13	Spending Statistics	4
1.3.14	Nearest Supermarket and Navigation	4
1.3.15	Multi-Device Synchronization	4
1.3.16	App Preferences (Theme, Font, Language)	4
2	Architecture Application	5
2.1	Overview	5
2.2	Data Model and Persistence	5
2.2.1	Core User Management	5
2.2.2	Shopping Lists and Purchased Items	5
2.2.3	Product Catalog and Categories	6
2.2.4	Supermarkets and Nearby Locations	7
2.2.5	Recipes and Receipts	9
2.2.6	Persistence and Synchronization (Offline-First)	9
2.3	Data Management	11
2.3.1	Firebase Firestore	11
2.3.2	Sync Engine (Firestore <-> SQLite)	11
2.3.3	SQLite (sqlite) Local Database	11
2.3.4	Shared Preferences	11
2.3.5	Firebase Auth + Google Sign-In	11
2.4	Device Services	11
2.4.1	GPS	12
2.4.2	Camera	12
2.5	External Services	12
2.5.1	Google Gemini 2.5 Flash	12
2.5.2	OpenStreetMap though Overpass API	12
2.5.3	Firestore	12
2.6	Dependencies	12
2.7	Widget Architecture	13
2.7.1	Responsive Design Strategy	13

2.7.2	Widget Organization	14
2.8	Sequence Diagrams	14
2.8.1	Authentication	14
2.8.2	Local-Remote Sync Engine Workflow	15
2.8.3	AI-Enhanced Product Categorization	16
2.8.4	AI-Driven Recipe Generation & Caching Flow	16
2.8.5	AI-Driven Price and Quantity Extraction	16
2.8.6	Nearest Supermarket	17
3	User Interface (UI)	18
3.1	UI Design Choices	18
3.1.1	Design Philosophy	18
3.2	Navigation Pattern	18
3.2.1	Home Tabs	18
3.2.2	Navigator	18
3.2.3	Secondary Features	19
3.3	Smartphone UI	20
3.3.1	Authentication	20
3.3.2	Lists tab	22
3.3.3	History tab	23
3.3.4	Register Shopping List Screen	24
3.3.5	Supermarkets tab	25
3.3.6	Statistics tab	26
3.3.7	Add recipe view	28
3.3.8	Trash view	28
3.4	Tablet UI	29
3.4.1	Adaptive Layout	29
3.4.2	Screen Adaptations	30
3.4.3	Navigation Enhancement	30
3.5	Accessibility	31
4	Testing Campaign	33
4.1	Testing Environment	33
4.2	Unit Testing	33
4.3	Widget Testing	33
4.4	Coverage Report	34
4.5	Integration Testing	34
4.6	User Testing	35
4.6.1	Testing Approach	35
4.6.2	Key Questions for Feedback	35
4.7	User Feedback Collection Template	36
4.7.1	Questionnaire Responses	36
5	Future Works	37
5.1	Wearable Device	37
5.2	Shared Lists	37
5.3	Supermarket APIs	37

1 | Introduction

1.1 Purpose

"List It" is a multi-device Android application designed to make grocery shopping more efficient.

The idea was born by evaluating the current methodologies that families use to gather their grocery and utility needs in a list. This process is cumbersome as it must take into account the preferences of multiple individuals in a group. The resulting shopping lists are messy and disorganized, making the actual shopping act inefficient. Moreover, tracking these expenses is complicated. It requires the usage of external apps or software, dealing with wasteful paper receipts and sometimes even the employment of accounting professionals.

"List It" solves these issues by having everything under control in a single place. It provides the end user with two main services. The automatic organization of shopping lists and the historical analysis of purchases.

The app employs LLMs to automatically associate products in a list to their respective categories and aisles in supermarkets. As a result, users are provided with tidy lists independently of the number or order of inserted products.

At shopping completion, users can avoid keeping with themselves any paper receipts by relying on the app's accounting feature. "List It" matches products in the shopping list with prices and quantities by taking a picture to the physical shopping receipt. This process leverages the usage of LLMs and Optical Character Recognition (OCR) to guarantee optimal matches. The user can then access this information on a dedicated screen in which expense analysis by time and category can be performed. As a consequence, the user will not have to worry about any other monetary management related to grocery shopping.

"List It" improves the overall shopping experience by limiting the preparation and effort that the user would otherwise have to employ to carry out such a common chore.

1.2 Requirements

The following list contains the functional requirements that the application shall satisfy.

1.2.1 Functional Requirements

- FR1** Users shall be able to access the app either by signing in/up or in a anonymous manner.
- FR2** Users shall be able to sign in/sign up in the app using email or Google SSO.
- FR3** Registered Users shall be able to logout from the app.
- FR4** Registered users shall be able to edit their personal profile information.
- FR5** Users shall be able to change the application language and appearance settings.
- FR6** Users shall be able to create/edit shopping lists.
- FR7** Users shall be able to delete shopping lists momentarily and permanently.
- FR8** Users shall be able to view shopping lists consistently.
- FR9** Users shall be able to select and access shopping lists consistently.

- FR10** Users shall be able to choose any available supermarket to associate to a selected shopping list.
- FR11** Users shall be able to manage supermarkets starting from a selected shopping list.
- FR12** Users shall be able to see the categories of the supermarket associated to a selected shopping list in the order specified for that supermarket.
- FR13** Users shall be able to input/delete/edit/mark any product in a selected shopping list.
- FR14** Users shall be able to change the category of inserted products in a list.
- FR15** Users shall be able to search and add AI generated recipes to a selected shopping list.
- FR16** Users shall be able to select specific products of AI generated recipes to add to the shopping list.
- FR17** Users shall be able to view all products previously added to a selected shopping list.
- FR18** Users shall be able to register any available shopping list.
- FR19** Users shall be able to view registered shopping lists consistently.
- FR20** Users shall be able to select and access registered shopping lists consistently.
- FR21** Users shall be able to insert price and quantity to products in a registered list.
- FR22** Users shall be able to take pictures of shopping list receipts.
- FR23** Users shall be able to edit/delete registered shopping lists.
- FR24** Users shall be able to view aggregated data related to their registered shopping lists by time and category.
- FR25** Users shall be able to modify the time period to analyze the aggregated data accordingly.
- FR26** Users shall be able to view detailed expenses per category within a selected time period for the aggregated data.
- FR27** Users shall be able to add/edit/delete supermarkets.
- FR28** Users shall be able to set any supermarket as their favorite.
- FR29** Users shall be able to view supermarkets consistently.
- FR30** Users shall be able to select and access supermarkets consistently.
- FR31** Users shall be able to change the order of categories in supermarkets.
- FR32** Users shall be able to view the categories in a selected supermarket in order and consistently.
- FR33** Users shall be able to add/edit/remove any category from a selected supermarket.
- FR34** Users shall be able to create/edit/delete categories.
- FR35** Users shall be able to access the nearest supermarket location at all times if available and allowed.

1.2.2 Non-Functional Requirements

- **Security:** User will be required to insert a password of a certain number of characters with the need to verify their emails through a verification link.
- **Compatibility:** The app should support iOS 13+, Android 5+, and iPadOS 13+.
- **Accessibility:** Dark and White Theme version with Italian and English translations available.

1.3 Features Implemented

1.3.1 Onboarding and Multi-Mode Access

On first launch, users can start without an account (anonymous access) or create a profile. Authentication supports email/password and Google sign in. The flow also handles upgrading an anonymous account to a permanent one.

1.3.2 Credential Recovery and Email Verification

A password reset flow is available via email with a cooldown to prevent abuse. For new subscribers and sensitive updates, the app includes an email verification step.

1.3.3 User Profile and Security

Users can update their profile name and manage credentials (email and password) with re-authentication. The UI correctly limits edits for accounts managed by external providers (e.g., Google).

1.3.4 Shopping List Management

The app allows users to create, rename and organize active lists with built-in search. Each list stores products, an optional supermarket, and modification timestamps to keep workflows consistent.

1.3.5 Product Entry with Smart Categorization

Products can be added manually or searched; when linked to a supermarket, they are automatically categorized by Gemini based on the available categories. Product–category associations persist per supermarket.

1.3.6 Supermarket and Category Management

Users can create and customize supermarkets, with editable and reorderable category lists. A favorite supermarket can be set up and is preselected for new lists.

1.3.7 Supermarket Adaptation

Every time the user initiates a new supermarket from the supermarket screen, the app will provide an already constructed supermarket based on the users previous supermarkets edit.

1.3.8 Shopping List Initialization

Every time the user creates a new shopping list, their favorite supermarket will be already preset in it.

1.3.9 AI-Assisted Recipe Generation

The app integrates an LLM service to generate ingredient lists and categories from a recipe name. Results can be edited (rename/remove ingredients) and added directly to a list.

1.3.10 Background Recipe Notifications

Recipe searches can run in the background; when finished, the app shows an in-app notification with quick access to the results, keeping the flow uninterrupted.

1.3.11 Purchase Registration and History

The lists can be “registered” at the end of a shopping session with quantities and prices per product. Completed lists are stored in a history view for later review.

1.3.12 Receipt Scanning with OCR and AI

The app uses on-device OCR to extract receipt text and an AI service to match quantities and prices to purchased items. Values are prefilled in the registration screen to reduce manual input.

1.3.13 Spending Statistics

Users can analyze spending by week, month, year, or custom range, with category breakdowns and product-level details to monitor budgets and habits.

1.3.14 Nearest Supermarket and Navigation

With location permissions, the app finds the nearest supermarket and opens maps for viewing or directions.

1.3.15 Multi-Device Synchronization

Data is stored locally and synchronized to the cloud via a push/pull sync engine with reconnection handling and remote updates. This ensures continuity across devices and sessions.

1.3.16 App Preferences (Theme, Font, Language)

Users can switch light/dark theme, adjust font size, and select language (IT/EN). The UI is responsive, with layouts optimized for phone and tablet.

2 | Architecture Application

2.1 Overview

This is a Flutter application designed as an offline-first shopping assistant. The UI is fully cross-platform and responsive (mobile/tablet), with localization, theming, and scalable typography managed in the presentation layer. Riverpod (AsyncNotifier/Provider) drives reactive state and side effects to keep the interface fluid even under heavy I/O.

The project follows a clean, layered architecture: *models* define domain entities, *repositories* expose data boundaries, *services* encapsulate device/external logic, and *providers* coordinate workflows across the app. All repositories have an abstract class definition, which is extended in the following way: real repositories, used for the real app; mock repositories, used in the widget tests to avoid calling external or slow services

Persistence is split between local and cloud storage: SQLite handles on-device data, while Firebase (Auth + Firestore) provides optional synchronization. A custom sync engine manages bidirectional updates with last-write-wins resolution and a `sync_box` queue that supports offline edits, cold-start delta pulls, and live snapshot updates.

External features are accessed through dedicated services: Google ML Kit for receipt OCR, Google Gemini for recipe search and product categorization, Overpass API and Geolocator for nearby supermarket discovery, and a map launcher for native navigation.

2.2 Data Model and Persistence

Data is implemented in Flutter models and stored locally in SQLite, with cloud sync through Firebase when the user is authenticated. Each entity exists locally by design, and a synchronization layer reconciles changes once connectivity is available.

2.2.1 Core User Management

The user system is built on Firebase Auth and a User model that stores the identifier (`uid`), email, and username, with `createdAt` and `lastModified` timestamps. Anonymous usage is supported for offline operation, while cloud synchronization is enabled only for authenticated users.

2.2.2 Shopping Lists and Purchased Items

The primary domain is centered on `ShoppingList`, representing a shopping list with name, creation date, optional supermarket, image, and lifecycle state (registered, in trash, deletion timestamp). Items in a list are modeled by `PurchasedProduct`, which binds a `Product` and a `Category` to a specific list, with price, quantity, `isBought`, and soft-delete (`isDeleted`) flags. The relationship is one-to-many: one list contains many purchased products (Figure 1).

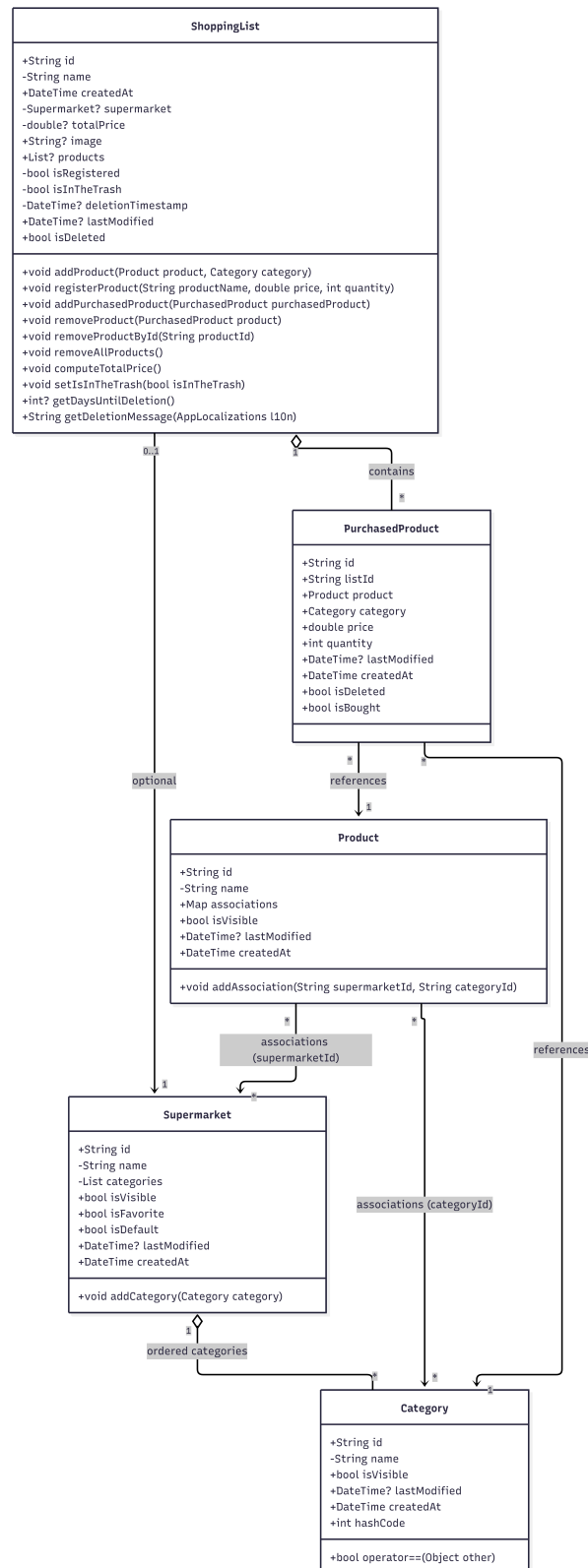


Figure 1: Shopping list and purchased product

2.2.3 Product Catalog and Categories

The global catalog consists of Product (id, name, visibility) and Category entities, both tracked with creation and modification timestamps (Figure 2). Product categorization is supermarket-

specific: each Product includes an associations map ($\text{supermarketId} \rightarrow \text{categoryId}$), also backed by a dedicated association table in the local database.

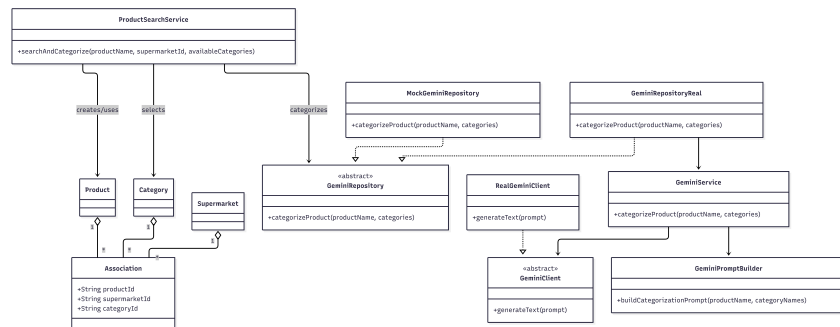


Figure 2: Product Catalog and Categories

2.2.4 Supermarkets and Nearby Locations

Supermarket describes persisted stores with associated categories, visibility, favorite, and default flags. On the other hand, for geolocation features, NearbySupermarket is a transient model obtained from location services, storing name, coordinates, and computed distance. This cleanly separates persistent store data from dynamic proximity data.

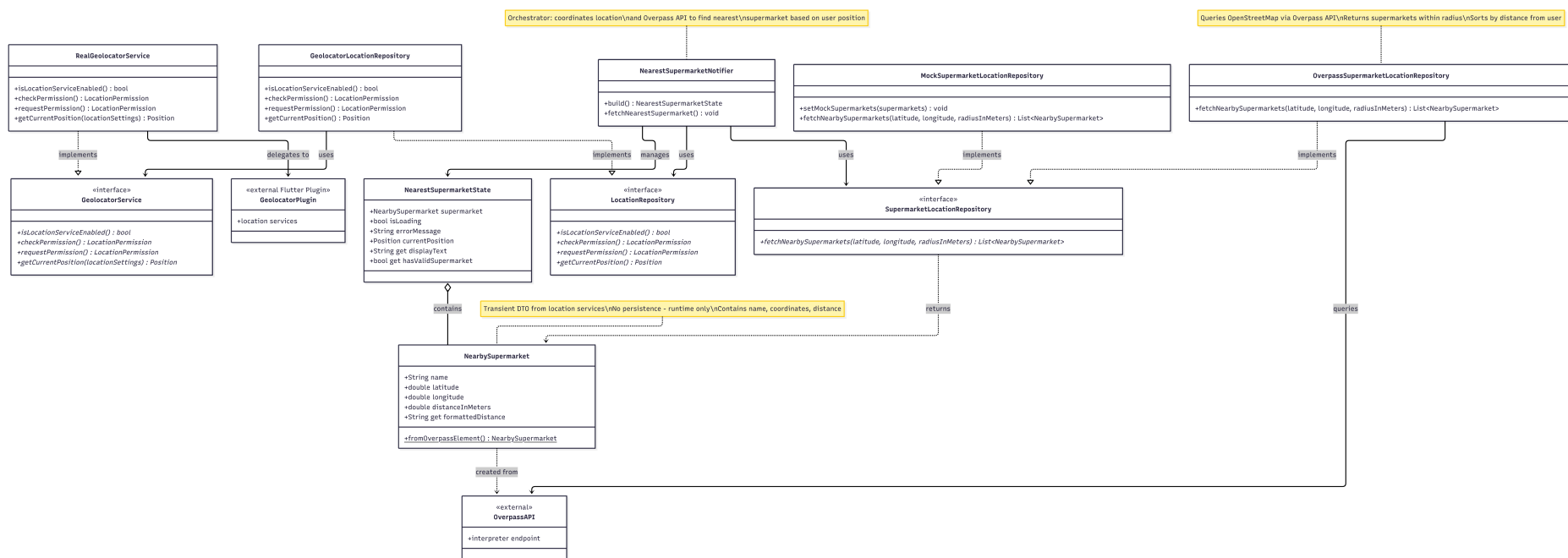


Figure 3: Nearest Supermarket

2.2.5 Recipes and Receipts

Advanced features are supported by `RecipeData`, which represents a generated recipe with product list, quantities, category labels, and recipe name, and is serialized to JSON for local cache (Figure 4). Receipt parsing uses the lightweight `ReceiptMatch` model to pair product identifiers or names with detected quantity and price.

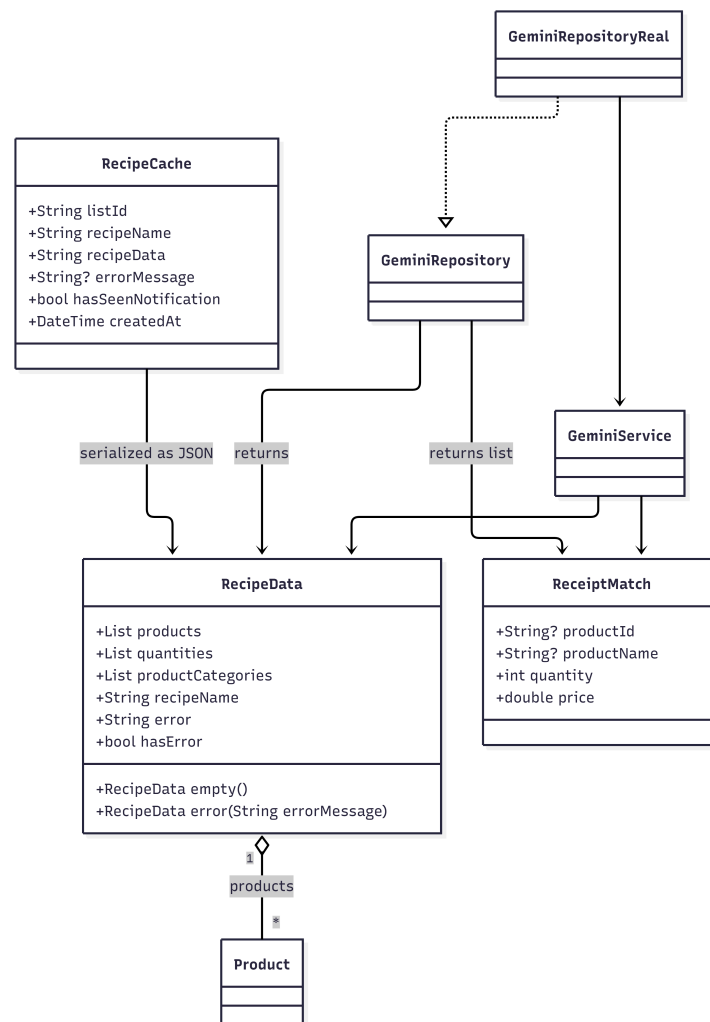


Figure 4: Recipes and Receipts

2.2.6 Persistence and Synchronization (Offline-First)

Local persistence relies on SQLite tables for lists, products, categories, supermarkets, associations, and recipe cache. Cloud synchronization uses Firestore plus a local `sync_box` queue. Pending operations are represented by `LocalSyncEntry` with `SyncOperation` (upsert/delete), using a last-write-wins strategy based on `lastModified`. The system supports:

- **Push**: propagating local changes when connectivity is available.
- **Pull**: cold-start delta sync using `lastSyncedAt` and live snapshot listeners.
- **Soft delete**: `isDeleted` to propagate removals without immediate local loss.

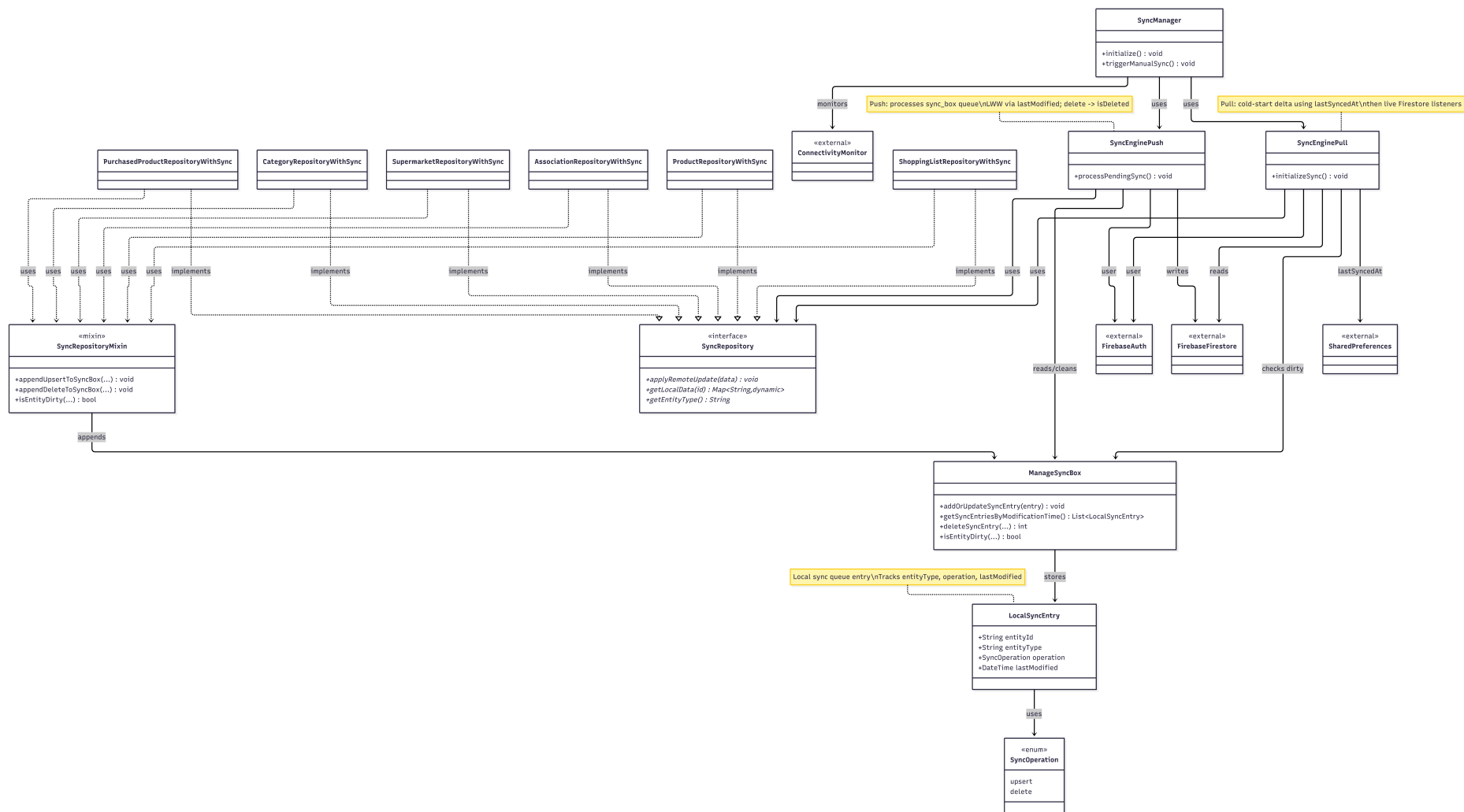


Figure 5: Sync Mechanism

2.3 Data Management

2.3.1 Firebase Firestore

Firebase Firestore is the primary cloud NoSQL database for user data: shopping lists, products, supermarkets, categories, associations, and purchase history. It provides low-latency access, automatic scaling, and tight integration with the rest of the Firebase stack. Documents are organized per user and synchronized through the sync engine.

2.3.2 Sync Engine (Firestore <-> SQLite)

Client-server data flow is handled by a custom Sync Engine that coordinates Local → Remote and Remote → Local synchronization.

- **Push:** local changes are queued in a `sync_box` and sent to Firestore using Last-Write-Wins conflict resolution.
- **Pull:** at cold start the app performs a delta sync based on `lastSyncedAt`, then enables live Firestore listeners for real-time updates.

This design keeps devices consistent and minimizes conflicts.

The main features implemented by the sync mechanism are:

- **Offline-First**
- **Silent-Updates**
- **Soft-Deletes**
- **Latest Write Wins**
- **Monotonic Timestamps for conflict resolution**

2.3.3 SQLite (sqlite) Local Database

SQLite is the main local database, used for offline operation and fast interactions. It stores all core entities (lists, products, supermarkets, categories, purchases, recipe cache) plus the `sync_box` table that tracks pending sync operations.

2.3.4 Shared Preferences

Shared Preferences is used for lightweight persistent state such as `lastSyncedAt`, local cooldowns (e.g., password reset), and small app flags. This ensures minimal state continuity across restarts and sign-outs.

2.3.5 Firebase Auth + Google Sign-In

Authentication is managed by Firebase Auth with support for email/password and Google Sign-In. The app also supports anonymous users, who work only on local data until they log in; after authentication, the Sync Engine uploads their local data to Firestore.

2.4 Device Services

Local device capabilities accessed via platform plugins and used by the app to implement its features:

2.4.1 GPS

Enables the *Nearest Supermarket* feature displayed in the top navigation bar. Geolocator requests GPS permissions and monitors the user's position, triggering updates when the user moves more than 100 meters.

2.4.2 Camera

Enables receipt photo capture for automatic text extraction and product registration. The camera initializes with high-resolution settings (JPEG format, back camera by default) and provides a live preview for framing receipt images. After capture, Google ML Kit Text Recognizer processes images locally on-device using Latin script recognition, extracting all visible text without sending data to external servers. Users navigate from the shopping list screen to the camera screen, photograph receipts, and the extracted text is used to automatically identify and add products to their shopping list.

2.5 External Services

2.5.1 Google Gemini 2.5 Flash

The application integrates Google Gemini Flash 2.5 through the `google_generative_ai` package to provide three core features: recipe ingredient discovery with automatic categorization, product categorization as items are added to shopping lists, and receipt text extraction for quantity and price matching. The service is accessed via a repository pattern initialized with a compile-time API key, executes recipe searches asynchronously in the background with local caching, and delivers results through user notifications. All Gemini interactions include robust error handling to ensure application stability when external service calls encounter failures.

2.5.2 OpenStreetMap through Overpass API

The application implements a location-aware feature that leverages OpenStreetMap data through the Overpass API to identify nearby supermarkets. Upon initialization, the app requests the user's GPS location and constructs an Overpass Query Language request to search for all shops tagged as supermarkets within a 5-kilometer radius. The Overpass API returns matching OSM nodes and ways with geographic coordinates, which the application then processes to calculate distances using the Haversine formula, identifying the nearest location. This supermarket information is displayed in the application's navigation bar and made interactive through integration with native map applications, with automatic updates triggered by user movement exceeding 100 meters or at five-minute intervals to maintain data freshness. When users tap the supermarket display, GPS coordinates are passed to platform-native map applications (Google Maps on Android, Apple Maps on iOS).

2.5.3 Firestore

The application leverages Firebase services through a hybrid local-first architecture with cloud synchronization. Firebase Authentication manages user identity, supporting anonymous access, email/password authentication, and Google Sign-In with account linking capabilities. Cloud Firestore serves as the remote persistence layer for user-generated content.

2.6 Dependencies

Here there is a table with the used Flutter packages and services (Table 1).

Package/Service	Purpose
flutter_localizations	Built-in localization delegates and locale support.
flutter_riverpod	State management and dependency injection.
isar	Model annotations and schema support.
firebase_core	Firebase initialization and platform setup.
firebase_auth	User authentication (email/password, Google sign-in).
google_sign_in	Google account sign-in flow.
cloud_firestore	Cloud Firestore database access and queries.
uuid	Unique ID generation for entities and sync entries.
sqflite	Local SQLite persistence for offline data and caches.
path	File path utilities (used to build database paths).
logger	Structured logging across services and repositories.
shared_preferences	Persistent key-value storage for settings.
intl	Localization and date/number formatting.
geolocator	Device geolocation.
http	HTTP client for remote API requests.
url_launcher	Opens map links in external apps.
connectivity_plus	Network connectivity monitoring for sync.
camera	Camera access for receipt capture.
google_generative_ai	Gemini API client for recipe/product intelligence.
google_mlkit_text_recogni	On-device OCR for receipt text extraction.

Table 1: Main Flutter dependencies used in the project

2.7 Widget Architecture

The application adopts a responsive design approach to ensure an optimal user experience across different screen sizes, with particular attention to smartphones and tablets. Its widget architecture is based on Flutter’s compositional paradigm, promoting modularity and component reuse.

2.7.1 Responsive Design Strategy

- **Mobile View** (< 600dp): Uses dedicated mobile screen classes (e.g., `HomeScreenMobileView`, `ListsScreenMobile`) with a bottom navigation bar, **IndexedStack** for single-pane tab switching, and stack-based navigation where details push as full-screen overlays.
- **Tablet View** ($\geq$ 600dp): Uses dedicated tablet screen classes (e.g., `HomeScreenTabletView`, `ListsScreenTablet`) with a persistent side navigation rail in a Row layout, and master-detail split views (40/60 ratio) displaying list and detail content side-by-side simultaneously.
- **Detection**: Screen size classification happens at runtime via `ScreenSize.isMobile()` /

`ScreenSize.isTablet()` using breakpoints, with device type determined once at launch and stored in `isTabletAtLaunch` for persistence decisions.

2.7.2 Widget Organization

- **Global Navigation Layer:** Shared components (`TopBarWithNavBar`, `SideMenu`) used across all screens with responsive navigation (bottom bar on mobile, side rail on tablet).
- **Responsive Utilities:** Helper classes (`ScreenSize`, `ResponsiveContainer`, `ResponsiveGridView`) provide breakpoint detection and adaptive layouts; `DetailPaneNavigator` manages isolated navigation within tablet split-view panes.
- **Reusable View Components:** Highly composable widgets (`SearchableShoppingListView`, `SearchableSupermarketsView`, `*GridView`) handle search, filtering, and responsive grid display, shared across Lists, History, and Supermarket screens.
- **Specialized UI Widgets:** Domain-specific components (`DraggableProductList`, `StatisticsPieChart`, `PasswordTextField`, `TrashDialog`) encapsulate feature-specific functionality with consistent styling.

2.8 Sequence Diagrams

2.8.1 Authentication

This sequence diagram (Figure 6) shows the authentication flow of the user. It takes into account the two different ways in which the user can sign in/up: either by email/password or by using the google provider. Furthermore, the possibility to use the app without being authenticated is also managed.

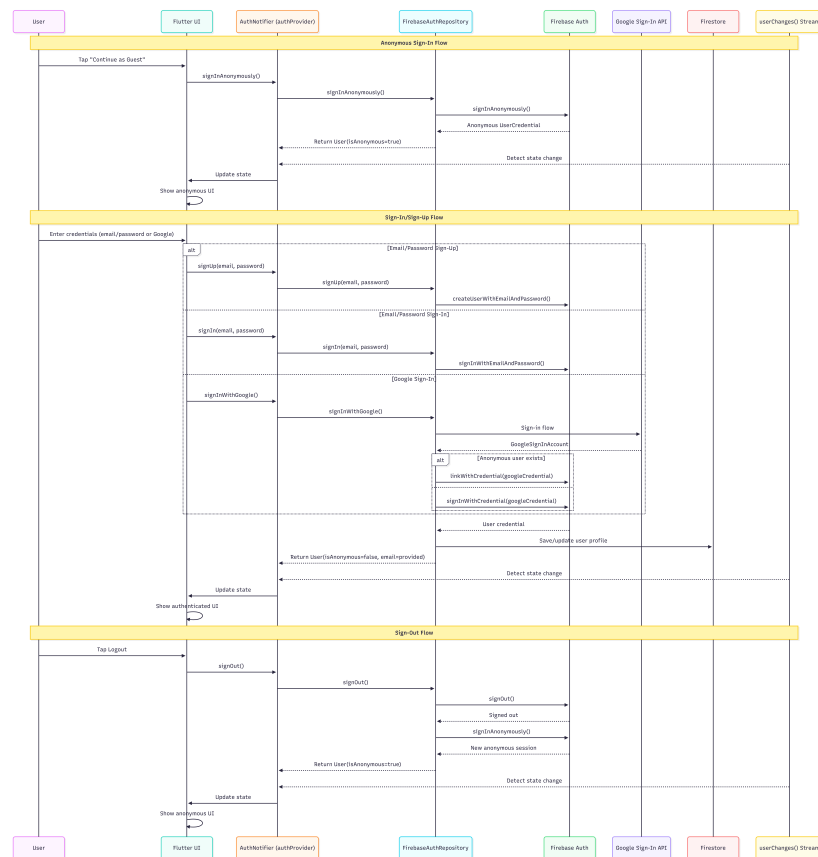


Figure 6: Firebase Authentication Flow

2.8.2 Local-Remote Sync Engine Workflow

This sequence diagram (Figure 7) illustrates the offline-first synchronization strategy for managing entity updates between a local SQLite database and Firebase Firestore.

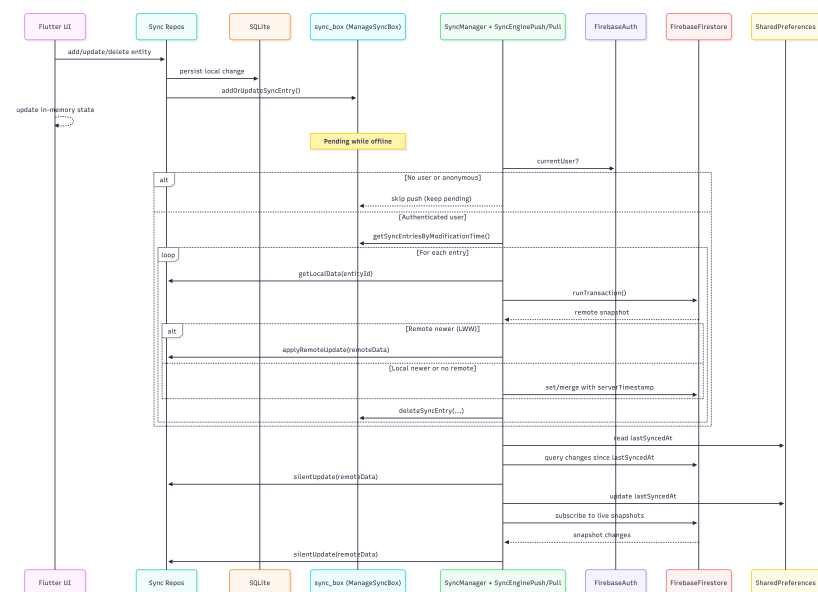


Figure 7: Local-Remote Sync Engine Workflow

2.8.3 AI-Enhanced Product Categorization

This sequence diagram (Figure 8) illustrates the workflow for adding or checking a product within the application, integrating Gemini AI for intelligent categorization and an offline-first synchronization strategy.

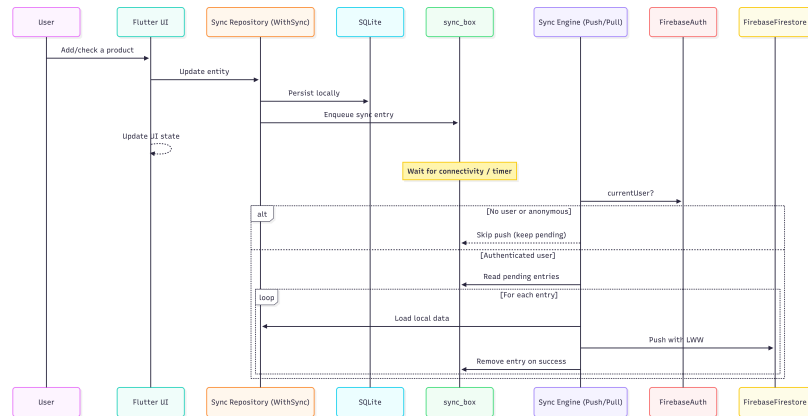


Figure 8: AI-Enhanced Product Categorization

2.8.4 AI-Driven Recipe Generation & Caching Flow

This sequence diagram (Figure 9) illustrates the workflow for retrieving and processing recipe data, integrating Gemini AI for automated parsing and a localized caching strategy.

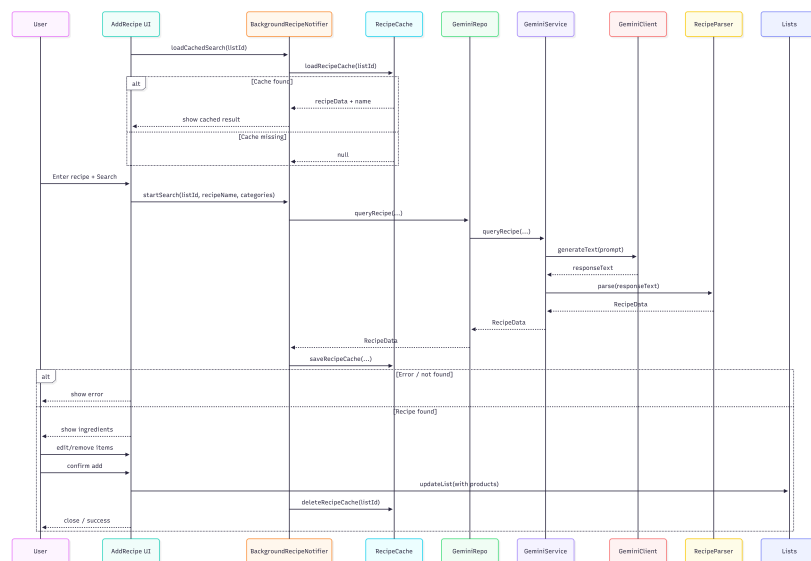


Figure 9: AI-Driven Recipe Generation & Caching Flow

2.8.5 AI-Driven Price and Quantity Extraction

This sequence diagram (Figure 10) illustrates the workflow for recognizing, matching and extracting price and quantity data in an automatic way from a receipt picture. The system is integrated by on device OCR offered by google_mlkit_text_recognition and Gemini AI for automated product and category matching.

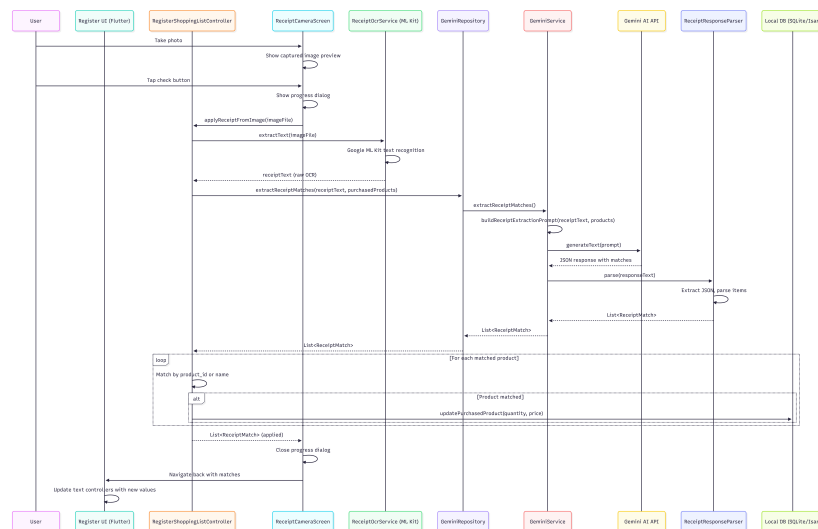


Figure 10: AI-Driven Price and Quantity Extraction

2.8.6 Nearest Supermarket

The following sequence diagram (Figure 11) reports the flow behind the implementation of the nearest supermarket feature. Such feature, when allowed and available, manages to always provide the user with information relative to the closest supermarket to them. The feature is able to gracefully withstand GPS unavailability and user movements.

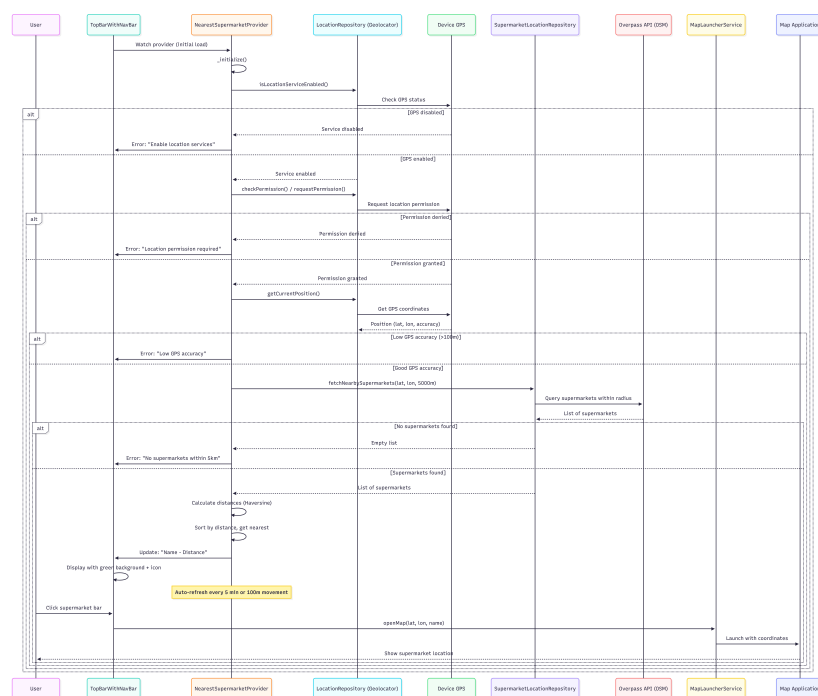


Figure 11: Nearest Supermarket Flow

3 | User Interface (UI)

3.1 UI Design Choices

"List It" is designed to provide the user with an intuitive and minimal UI. Widgets and screens are built in order to reduce complexity and open the possibilities to people from every background to interact with it. Several customization options are further provided to make user adapt the UI to their personal likings.

3.1.1 Design Philosophy

Ease of use is implemented by separating concerns among several screens and relying on top of natural navigation.

Two different UI layouts further guarantee ease-of-use with respect to the device the app is running on top of. The app can adapt itself in a responsive manner when either running on a mobile or a tablet device.

3.2 Navigation Pattern

The Navigation Pattern is designed in a tree-like manner. The root screens are represented by the app's home tab pages accessible by a Navigator Bar. From them, every branch is built by navigating through screens using the Flutter's Navigator object.

Branches implement the logical navigation flow across screens in a natural and smooth manner. By following the sequence of screens the user can fully gain access to the offered app's services.

Each branch (i.e. navigation flow) either terminates on a leaf-node (i.e. a dead-end screen) or concludes going back to one of the four starting roots (i.e. home tabs).

3.2.1 Home Tabs

The primary navigation (across root nodes) uses a bottom navigation bar for the main functionality, allowing fast access (Figure 12). From it the Navigation bar structure:

- **Lists:** active shopping lists and creation of it
- **History:** registered/completed lists with prices and quantities
- **Supermarket:** supermarket management and categories customization
- **Statistics:** spending and category insights.

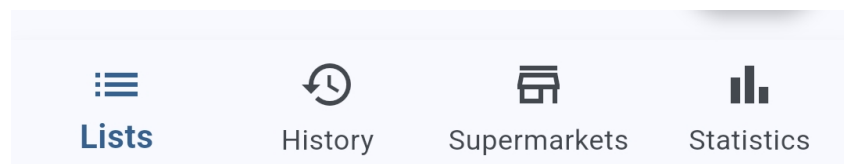


Figure 12: Bottom nav bar

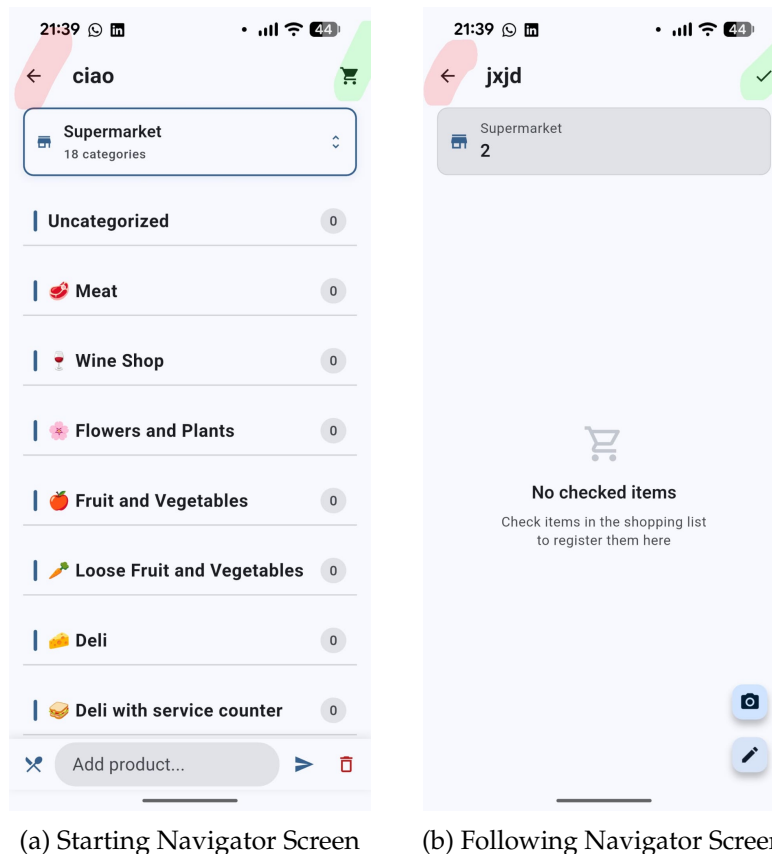
3.2.2 Navigator

The logical navigation through screens (tree branches) is implemented by the Flutter's Navigator object. The application of such object is characterized by the presence of a "back-arrow" button

displayed in the top-left corner. From any given screen in a branch, the user has a few other available interactions to take to gain access to the next screen in the branching logical sequence (e.g. by pressing a button).

Hereunder, there is reported one example of application of the Navigator object. In red there are highlighted the Navigator's characterizing 'back-arrow' buttons. In green the interaction that the user can take to go from the current screen to the next one in the sequence.

In the example below if the user presses the cart button, in the left-end screen, they get redirected to the left-end one.



3.2.3 Secondary Features

Finally, a side menu (Figure 14) is implemented to allow to access secondary features:

- **Profile:** user information, like email, password...
- **Settings:** change between light/dark mode, language and font size choice
- **Trash:** shopping lists permanent deletion

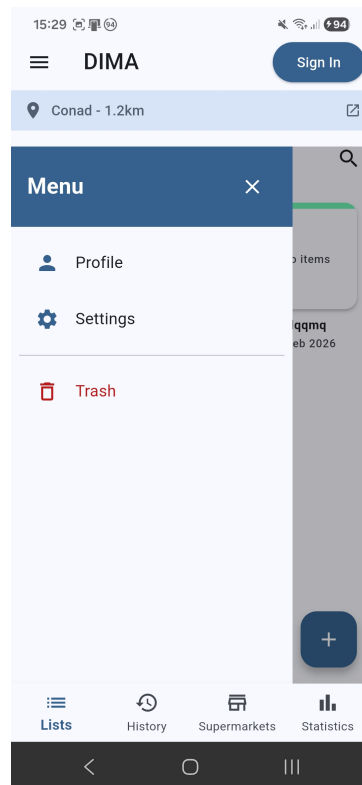


Figure 14: Side menu

3.3 Smartphone UI

3.3.1 Authentication

The application supports multiple authentication methods to accommodate different user preferences and use cases (Figure 6).

Anonymous Access: First-time users are presented with an initial welcome screen offering two options: creating an account or continuing anonymously. Anonymous users can explore the app's features immediately and later upgrade their account to preserve their data.

Email/Password Authentication: The sign-up form requires a username, email address, and password (minimum 8 characters). Upon successful registration, users are prompted to verify their email address before full account activation. The sign-in form provides a streamlined login experience with validation feedback and a "Forgot Password" option for account recovery.

Google Sign-In: Users can authenticate using their Google account, providing a one-tap authentication experience. This method automatically handles account linking for users upgrading from anonymous access.

Form Switching: The welcome screen allows users to toggle seamlessly between sign-up and sign-in forms without navigation, reducing friction in the authentication flow. Input validation provides immediate feedback for invalid credentials or missing required fields.

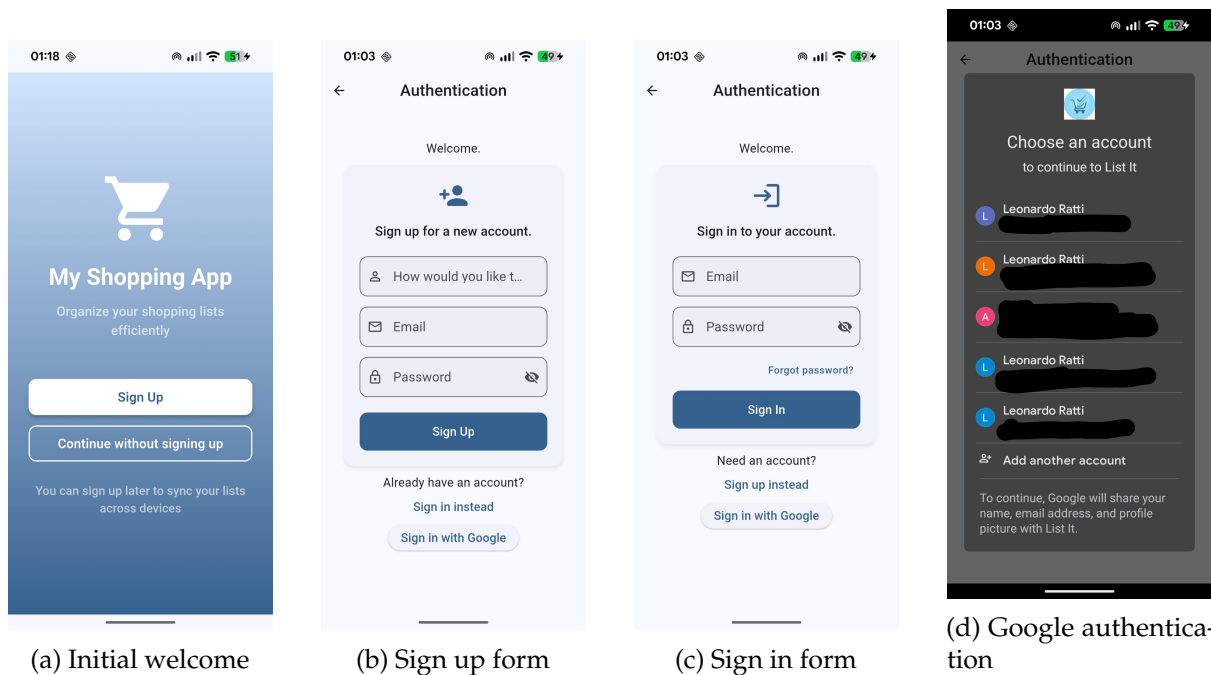


Figure 15: Authentication flow: account creation and sign-in options.

Password Recovery: Users who forget their password can request a password reset link via email. The forgot password screen requires email confirmation to prevent typos and implements a cooldown mechanism to prevent abuse, displaying a countdown timer when the rate limit is active (Figure 16).

Email Verification: After signing up with email credentials, users are directed to a verification screen that displays the registered email address and provides options to resend the verification email or check verification status. The screen includes clear instructions and visual feedback for the verification process.

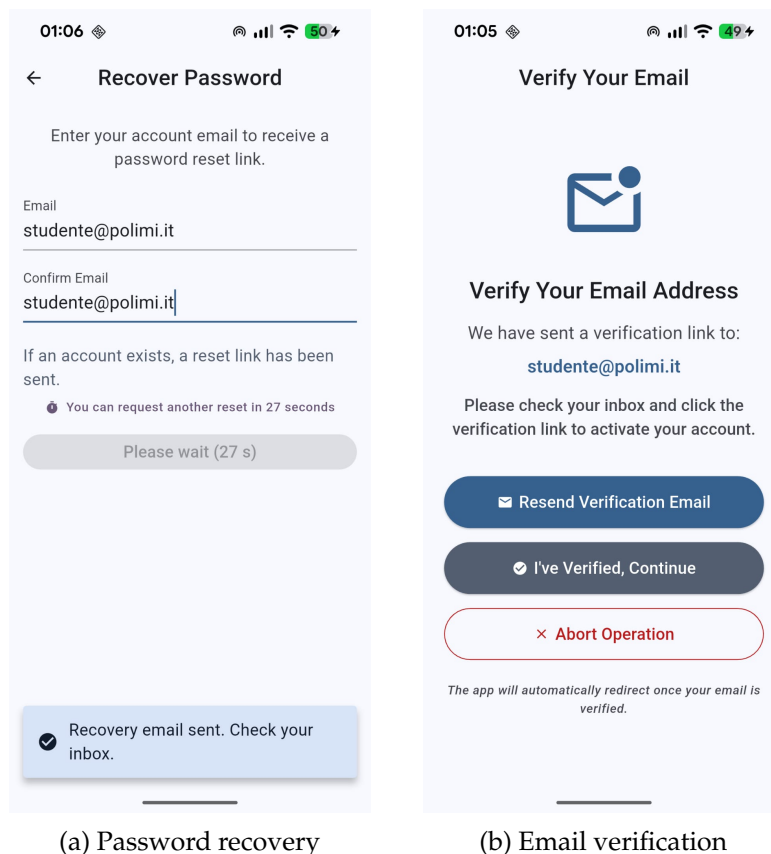


Figure 16: Account recovery and email verification features.

3.3.2 Lists tab

The main view displays all active shopping lists as compact cards, each showing the list name, creation date, and a brief preview of items (Figure 17).

List Search and creation: A search action allows fast filtering by list name, while a floating action button supports instant list creation.

Selection and Bulk Deletion: Users can enter a multi-selection mode to mark several lists at once. Selected cards show a check indicator, and a confirmation dialog is required before deletion. The deleted lists will be put in trash.

Inline List Renaming: List titles can be edited directly within the card view.

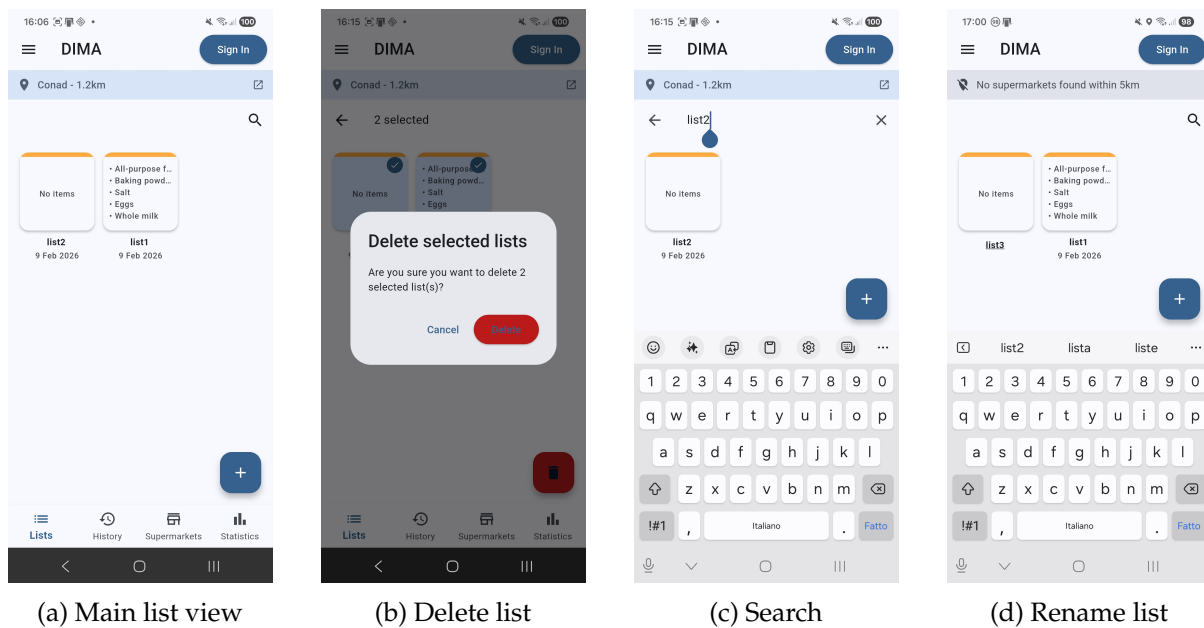


Figure 17: Lists tab: management and organization features.

3.3.3 History tab

The history tab displays all registered shopping lists—those that have been finalized after a shopping trip. Each list is presented as a compact card showing the list name, creation date, and status information (Figure 18).

List Search: A search action enables quick filtering of registered lists by name, making it easy to locate past shopping trips.

List Access: Tapping on any registered list navigates to the Register Shopping List screen, where users can view the recorded quantities and prices, make adjustments, or export the data.

List Organization: Lists are automatically sorted by creation date (newest first), providing chronological access to shopping history for budget tracking and analysis.

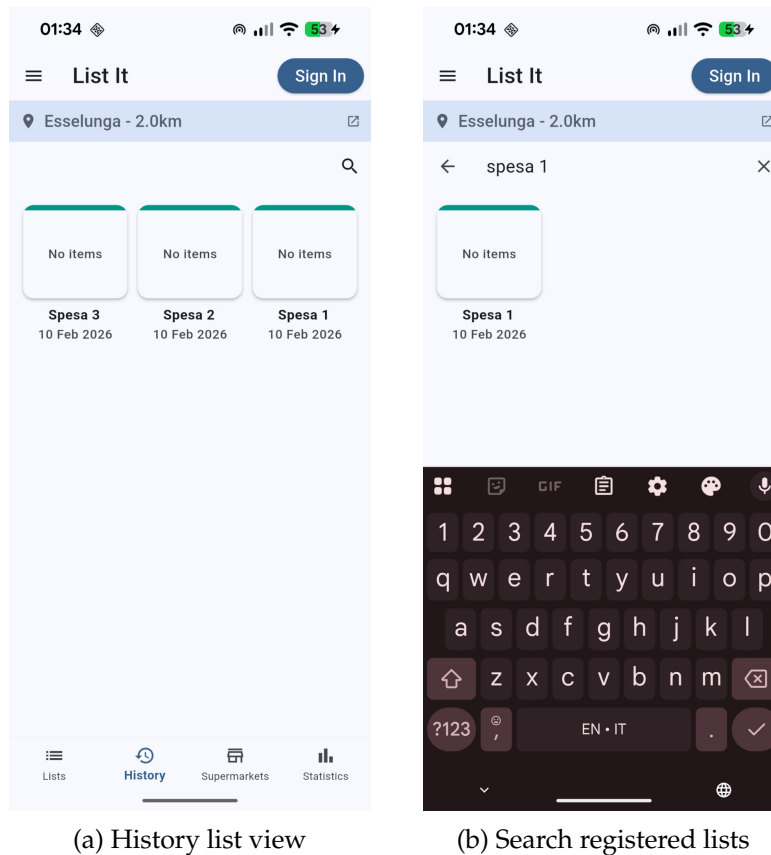


Figure 18: History tab: access to registered shopping lists.

3.3.4 Register Shopping List Screen

The Register Shopping List screen serves as the finalization interface where users record actual purchase details after completing their shopping trip. This screen transitions a shopping list from an active planning state to a registered historical record (Figure 19).

Supermarket Display: A fixed, non-interactive card displays the selected supermarket for the shopping trip, providing context for the purchase records.

Product Registration: Only products marked as bought (checked) in the list are displayed, organized by category. Each product features two input fields for recording the actual quantity purchased and the price paid. Text fields support numeric input with appropriate formatting (integers for quantity, decimals for price).

Receipt Scanning: A camera floating action button enables users to scan paper receipts using OCR (Optical Character Recognition) technology. The system automatically extracts product names, quantities, and prices from the receipt image, pre-filling the corresponding fields to minimize manual data entry and reduce errors.

Finalization Actions: Three primary actions control the registration workflow:

- **Back button:** Saves any quantity and price modifications to the database without registering the list, allowing users to return later.
- **Check button:** Finalizes the registration by marking the list as registered (sets `is_registered=true`), auto-filling quantity values of 1 for any bought products without specified quantities, and navigating to the History tab.

- **Pencil button:** Unregisters the list (sets `is_registered=false`) and reopens it in the standard list detail view, enabling users to add or remove products or make other structural changes before re-registering.

Empty State Handling: When no products have been marked as bought, the screen displays an informative message prompting users to check items before attempting registration, ensuring data quality and preventing empty registered lists.

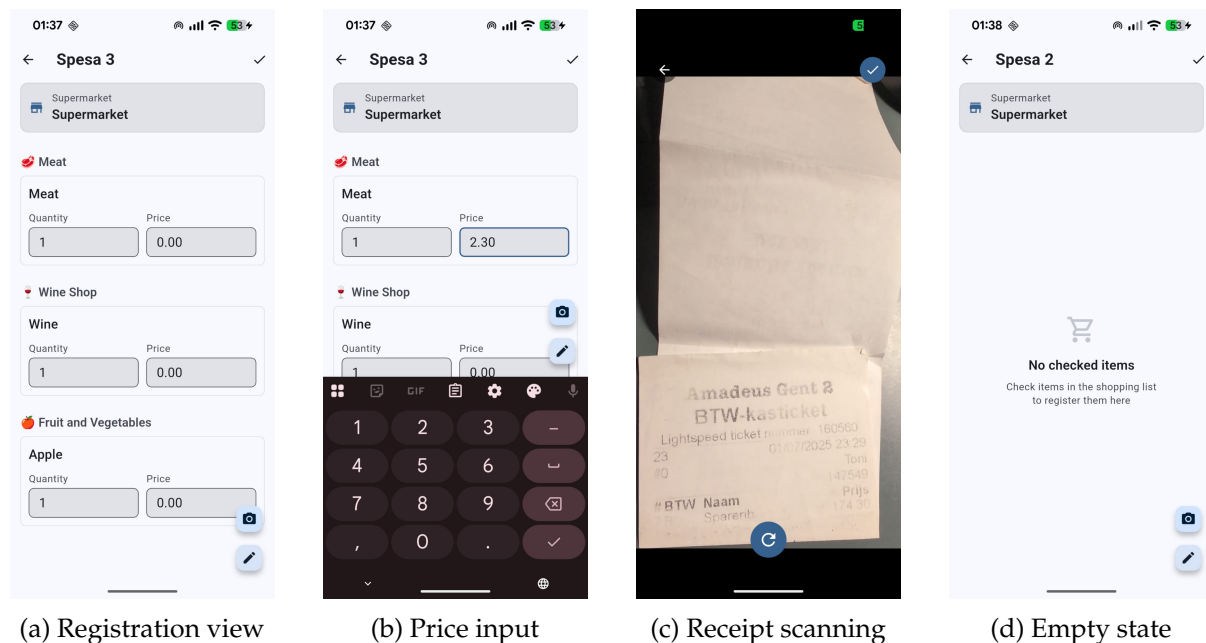


Figure 19: Register Shopping List screen features.

3.3.5 Supermarkets tab

The main view displays all visible supermarkets organized with favorites prominently featured and others in alphabetical order. Each supermarket is presented as a card showing the supermarket name and a visual indicator of its favorite status. The interface supports efficient supermarket management through search, creation, customization, and organization capabilities.

Supermarket Discovery and Creation: A search action enables quick filtering by supermarket name, while a floating action button supports instant supermarket creation. Both operations are optimized for efficient management of multiple supermarket locations.

Favorite Management: Users can mark supermarkets as favorites using a star icon. Favorite supermarkets appear at the top of the list, followed by others in alphabetical order, providing quick access to frequently used locations.

Supermarket Customization: When creating or editing a supermarket, users can set the name and manage categories. In creation mode, the new supermarket inherits categories from the last edited supermarket for consistency and speed.

Category Management: Within a customized supermarket, users can add, remove, reorder, and edit product categories. Categories are displayed as draggable cards—users can drag them to reorder or tap to edit custom settings. The uncategorized category is automatically maintained and hidden from the user interface. A visual message appears when no categories are assigned, prompting the user to add them.

Data Persistence and Deletion: Changes to supermarket information are persisted when the user confirms with a checkmark action. Deletion is protected by a confirmation dialog to prevent accidental removal. The application provides clear feedback for all operations through snackbar notifications.

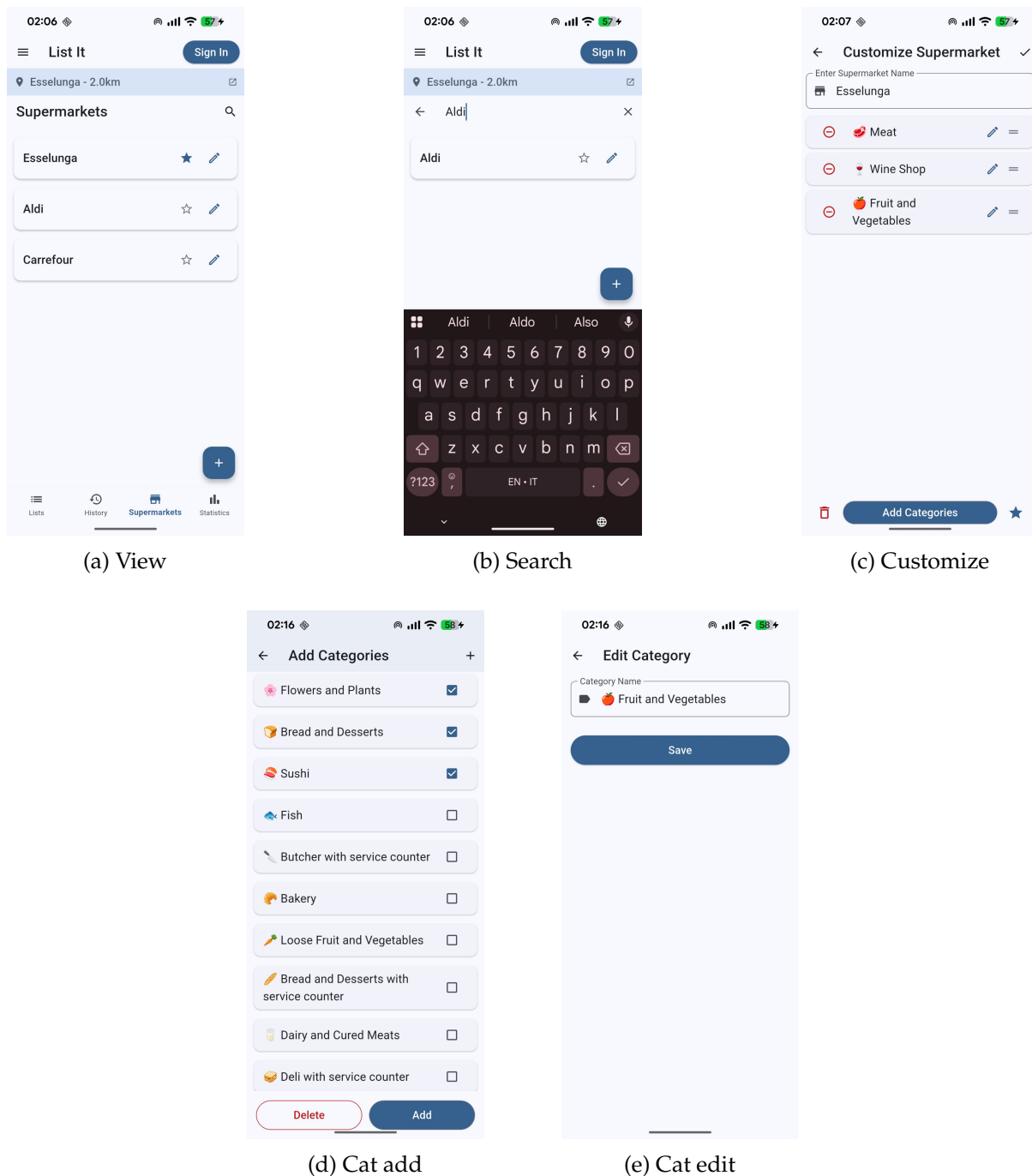


Figure 20: Supermarkets tab features.

3.3.6 Statistics tab

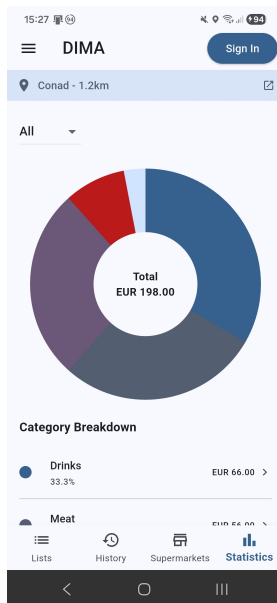
The Statistics Tab transforms past shopping data into clear spending insights, combining a visual summary with actionable detail (Figure 21).

Period Selection: A compact dropdown lets users switch between All, Week, Month, Year, and Custom ranges. The chart and list update immediately, enabling quick comparisons across time

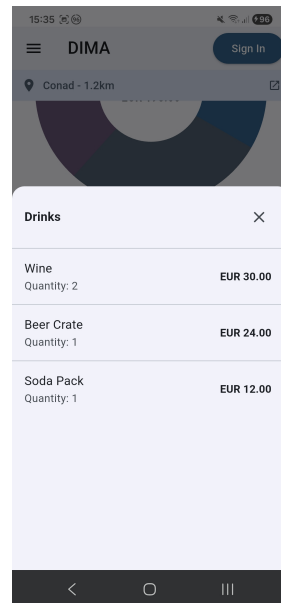
windows.

Spending Overview: A donut chart displays the total amount in the center, with colored segments representing category shares. This provides an at-a-glance understanding of where most budget is spent.

Category Breakdown and Drill-Down: Below the chart, categories are listed with their percentage and total cost. Tapping a category opens a bottom sheet showing the individual products with quantities and prices, allowing detailed review without leaving the tab.



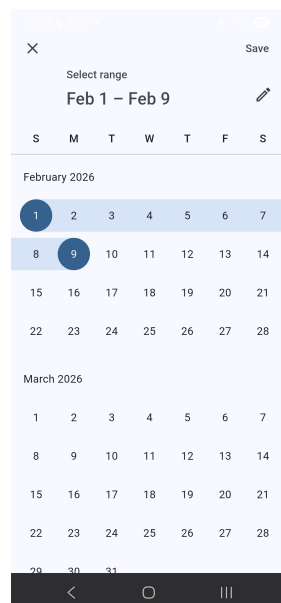
(a) Main view



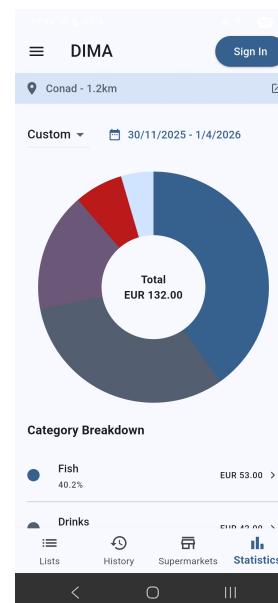
(b) Product list



(c) Period selection



(d) Range selection



(e) Range view

Figure 21: Statistics tab: comprehensive overview of features.

3.3.7 Add recipe view

The Add Recipe view supports recipe search using the Gemini API, giving as a result the ingredients list (Figure 22).

Recipe Search Input: Users enter the recipe name in a dedicated text field and trigger a search with a prominent button.

Result Preview and Editing: When a recipe is found, the screen presents a compact recipe card followed by an ingredient list. Each ingredient shows its name and quantity, and can be removed or adjusted before import. Next to each item, there is its categorization.

Error Handling and Feedback: If the recipe is not found, the view shows a clear warning message and invites the user to try again. Both successful and non-successful searches also trigger feedback (which will be displayed even if we are in other screens) to confirm the operation. We implemented an in-app notification system to handle the recipe search process, which can be time-consuming. This approach ensures a non-blocking user experience, allowing users to navigate other sections of the app and perform different tasks while the search continues in the background.

Add to List Action: A bottom button adds the selected ingredients to the current shopping list.

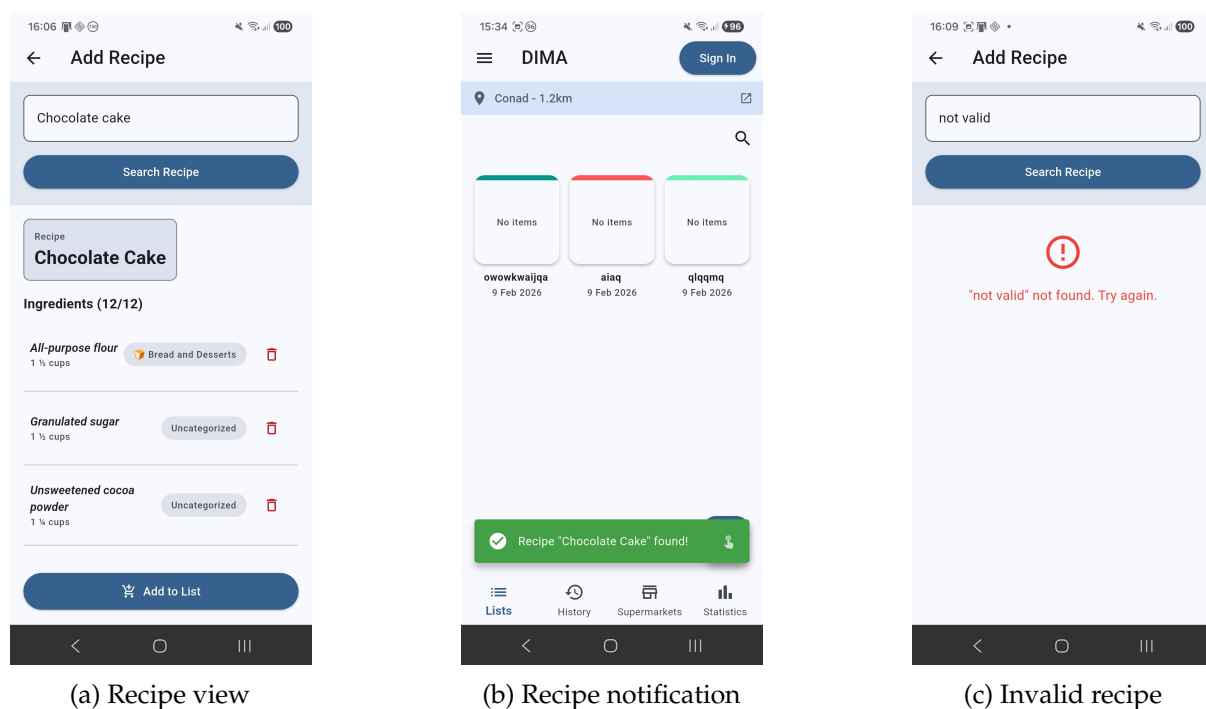


Figure 22: Recipe view: comprehensive overview of features

3.3.8 Trash view

The Trash view provides a safe recovery space for deleted lists, reducing accidental loss while keeping the main workspace clean (Figure 23).

Soft Deletion and Countdown: Deleted lists are moved to the Trash instead of being removed immediately. Each entry shows a countdown (e.g., “Delete in 30 days”), communicating the remaining time before permanent deletion.

Restore and Permanent Deletion: Users can restore individual lists with a single action or remove them permanently. A global “Empty trash” action clears all items at once, while “Restore all”

recovers every list in the trash.

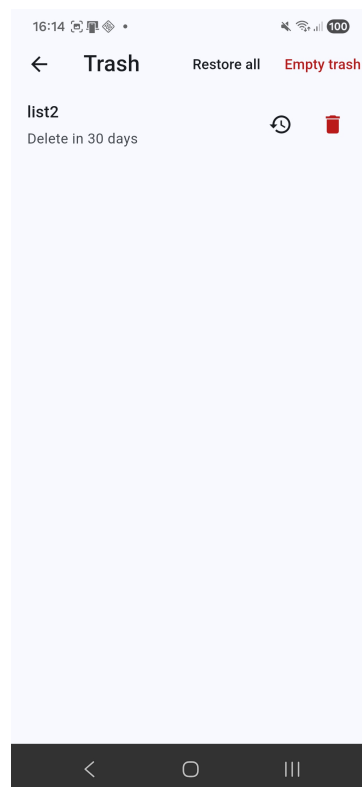


Figure 23: Trash view

3.4 Tablet UI

The app manages to adapt its UI according to screens of different sizes. The result is an improved user experience with respect to the user's device dimensions while providing the same features and services.

3.4.1 Adaptive Layout

- **Adaptive Strategy:** The app measures the device size on first usage and fixes the layout parameters according to it. These parameters manage to properly load either the tablet views or mobile views in a consistent manner throughout the app life cycle on a device.
- **Multi-Pane Architecture** The main difference of the tablet view with respect to the mobile views consists in a master-detail pane architecture. On tablets, the user will be able to find the home tabs in the master pane on the left side. On the right-end, the user will be able to interact it with the logical navigation through screens.
- **Navigation Enhancements:** To leverage the different size further, the logical navigation through screens has being adapted to allow for a smooth user experience even on larger screens.
- **Navigation Rail:** The second main tablet view with respect to the mobile view consists in a navigation side rail instead of a bottom navigation bar. This layout improvement allows the user to have direct access to the home tab without having to reach the bottom of a larger screen.

3.4.2 Screen Adaptations

Hereunder there are visual examples related to the enhancements in the tablet layout. These include the master-detail screen split, the rail bar instead of the navigation bar and a dedicated side panel for recipe search instead of a full screen for them.

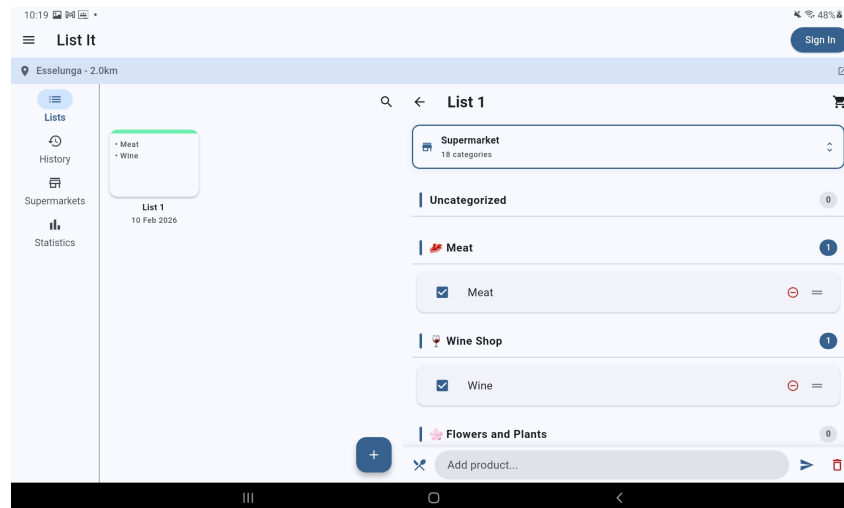


Figure 24: Master-Detail Pane with Side Rail

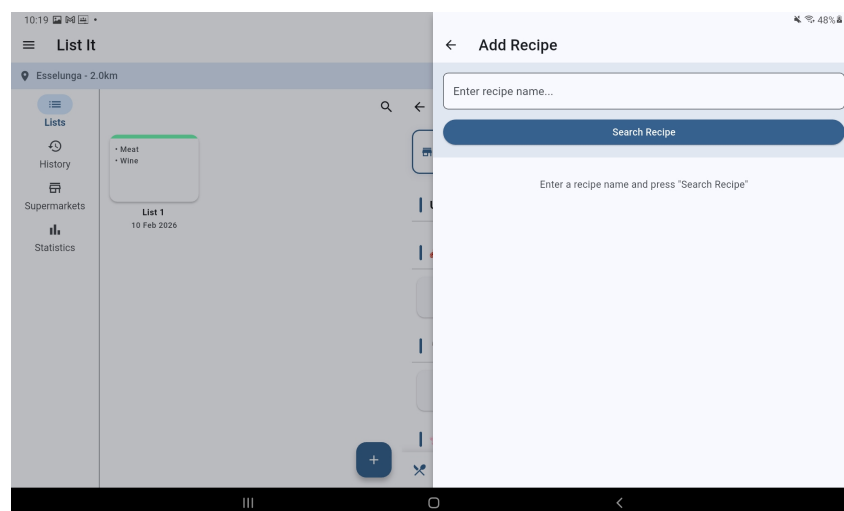


Figure 25: Recipe Panel

3.4.3 Navigation Enhancement

- **DetailPaneNavigator:** Every navigation through screens takes place in the detail pane of the Tablet layout. To enforce this navigation a `DetailPaneNavigator` widget has been employed. Such widget wraps the detail-pane side of the screen with a restricted `Navigator` object. Such object, makes it possible for the navigation through screens to take place in the right side panel leveraging the already implemented UIs for mobile devices. This approach achieves a balance between user usability and code recycling, without having to implement different tablet views from scratch.
- **Logical Navigation:** Logical navigation through screens has also been improved to adapt to the master-detail pane view. Specifically, the navigation flow between the detail list

screen and the register list screen has been modified. On tablets the user will not be directly redirected to the history home tab when registering a list. Instead, they will be kept on the list home tab while the list gets properly moved. The same happens in an opposite manner when a user unregisters a list to make it editable once again. This allows to keep a consistent view of home tabs, avoiding abrupt changes without having the user pressing any button.

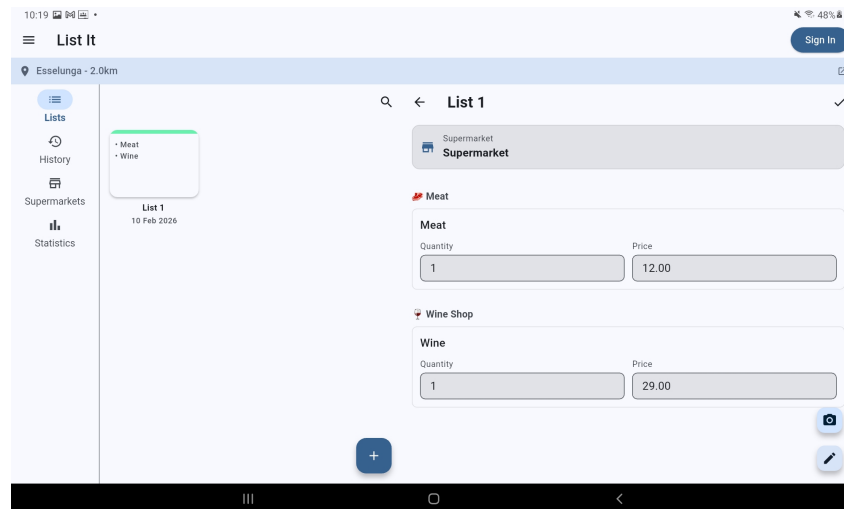


Figure 26: Before Registration

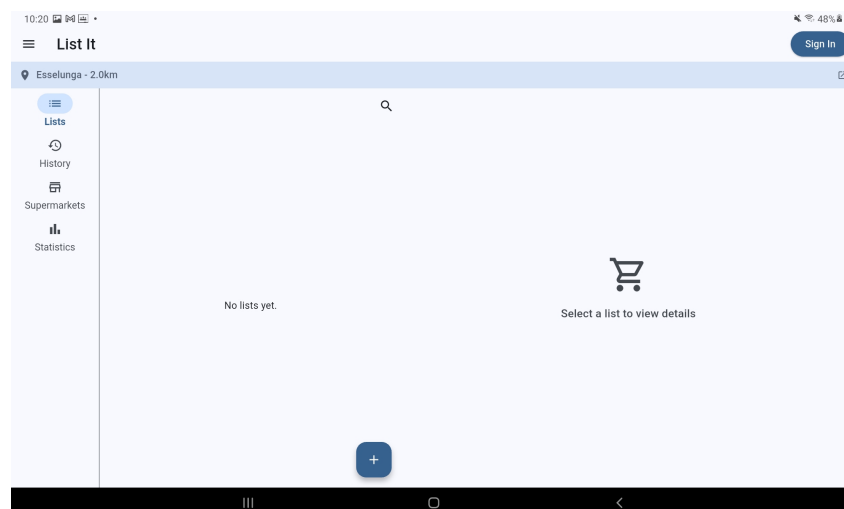


Figure 27: After Registration

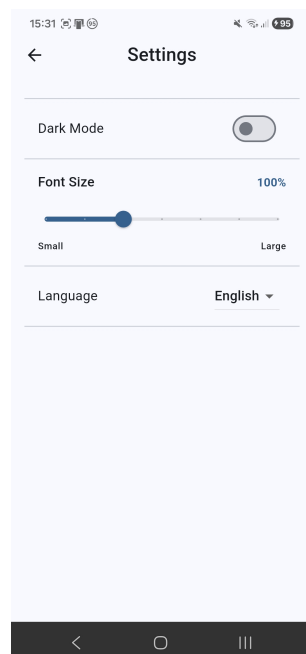
3.5 Accessibility

This app includes accessibility options to enhance the user experience and accommodate diverse needs (Figure 28).

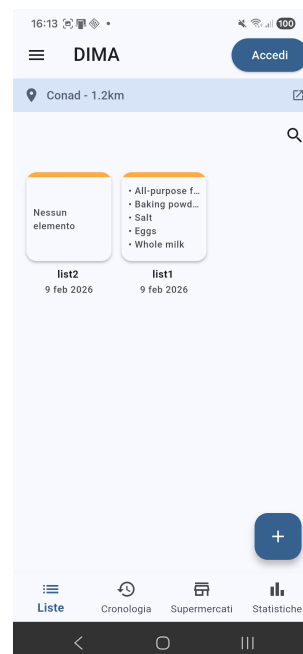
Dark Mode Toggle: Users can switch between light and dark themes with a single toggle. This improves readability in low-light environments and reduces eye strain during extended use.

Font Size Scaling: A slider lets users increase or decrease the global text size. A percentage indicator provides immediate feedback, ensuring the chosen size is clear and consistent across the app.

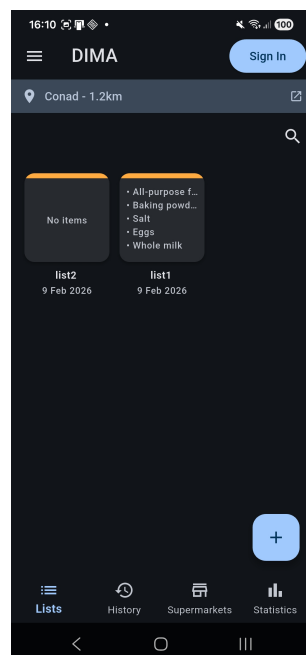
Language Selection: The interface supports multiple languages (English and Italian). Users can change the app language directly from Settings using a dropdown, enabling localization without restarting the app.



(a) Settings view



(b) Language support



(c) Lists tab dark



(d) Statistics tab dark

Figure 28: Accessibility features

4 | Testing Campaign

4.1 Testing Environment

Testing for DIMA is conducted in a *Mock environment*: external services (Firebase Auth, Firestore, and any network-based features such as recipe search or receipt parsing) are replaced with mock implementations. This allows deterministic tests without network dependencies. Mocks are injected through Riverpod provider overrides, following a dependency injection approach.

4.2 Unit Testing

Unit tests focus on core logic that can be validated in isolation:

- **Data Models:** Correct serialization/deserialization and data conversion for shopping lists, products, supermarkets, and categories.
- **State Management:** Riverpod providers and notifiers (lists, supermarkets, settings) are validated with mocked dependencies.
- **Business Logic:** Operations such as list registration, trash retention rules, category assignment, and statistics calculations.
- **Utilities:** Date formatting, category localization, and helper methods used across screens.

Because several workflows depend on cloud services, unit testing is concentrated on data handling and local state transitions.

4.3 Widget Testing

Widget tests verify that UI components behave correctly with mocked providers:

- **Lists and History Screens:** Rendering of list cards, empty states, and selection indicators.
- **Dialogs and Inline Editing:** Rename fields, delete confirmation dialogs, and validation feedback.
- **Statistics UI:** Period selector, chart rendering triggers, and category drill-down sheet.
- **Settings Screen:** Theme toggle, font size slider, and language dropdown interactions.

These tests ensure that key screens respond correctly to state changes without requiring full backend connectivity.

4.4 Coverage Report

Coverage by Folder

Folder	Covered Lines	Total Lines	Coverage
lib/l10n	242	731	33.11% 
lib/models	452	459	98.47% 
lib/providers	619	813	76.14% 
lib/repositories	1040	1412	73.65% 
lib/screens	2722	3886	70.05% 
lib/services	1101	1240	88.79% 
lib/utills	262	290	90.34% 
lib/widgets	974	1222	79.71% 

Figure 29: Code Coverage Analysis by Directory

Our current testing suite reflects a strategic prioritization of the codebase. We have focused our efforts where they provide the most long-term stability and reliability for the application.

Unit tests: we can see a high coverage in the model, repositories, providers, services, utils folders. These are the most important parts because it's the core logic of the application and so we focused on it to prevent future errors in eventual new implementations.

Widget tests: we can see a slightly lower coverage in screens and widgets folder. These are the parts that change frequently, and so we focused on main user interaction instead of focusing on every single detail in the screens.

Localization (l10n): localization present the lowest coverage as it contains only autogenerated labels and the text translated in more languages.

4.5 Integration Testing

To conclude the testing suite, integration testing was the last level of testing that was applied. These tests manage to properly assess the navigation flow between multiple screens and the correct workings of feature requirements.

In order to guarantee robustness on the most important services offered by the app, six integration tests have been designed. These tests cover the main workflows from each one of the home tabs, ensuring their reliability.

Tests

- **Authentication:** Tests the complete authentication flow including login, registration, password reset, email verification, and ensures that authenticated users can access the home screen with their shopping lists data.
- **Category Persistence:** Verifies that product category changes persist correctly in the database by creating a list, adding a product, changing its category, and confirming the newly associated category remains after closing and reopening the list.
- **List Persistence:** Tests that shopping lists and their products persist in the database, including creating lists, adding products, toggling their bought status, and verifying all data persists after closing and reopening the app.

- **List Registration:** Validates the list registration feature by creating a list, adding products with quantity and price information, registering the list, and confirming that all entered data persists after reopening.
- **Statistical Updates:** Tests that product statistics are correctly updated and persisted when a list is registered with price information. It verifies that the statistics home tab displays the product details after reopening the app.
- **Supermarket Management:** Comprehensive test for supermarket management including creating a supermarket, adding and reordering custom categories, setting it as favorite, and ensuring all configurations persist across app sessions.

Integration tests succeed in providing reliability with respect to the app's fundamental services. This allows to improve the guarantees to end users with respect to the app correct behavior.

With integration tests, the testing suite is complete. Future integrations and extensions can rely on the present testing suite to ensure a constant correct behavior of the app while performing further developments.

4.6 User Testing

In order to guarantee a robust user experience, the app was further tested by an experimental and practical interaction of a user with the app's "beta" version.

4.6.1 Testing Approach

The user involved was given 48 hours of free trial of the app. The user was first introduced to the idea and to the app purpose. After such time passed, the user's feedback was collected through a structured questionnaire. This allowed to have a proper idea of how the app will perform once deployed to real public.

4.6.2 Key Questions for Feedback

The following open-ended questionnaire shall be submitted to users after a continuous usage period of approximately 48 hours. Its purpose is to collect qualitative feedback regarding usability, functionality, and perceived value of the application.

1. **Overall experience**

After using the application for approximately 48 hours, how would you describe your overall experience? Please mention what you found most positive and what you found most problematic.

2. **Usability and navigation**

How easy was it to understand and navigate the application's main features? Were there any interactions, screens, or workflows that felt confusing or unintuitive?

3. **Functionality and reliability**

Did the application behave as you expected during your usage period? Please describe any bugs, crashes, performance issues, or unexpected behaviors you encountered.

4. **Usefulness and value**

In your opinion, how useful is the application for its intended purpose? Did it help you accomplish your tasks efficiently, and why or why not?

5. **Improvements and suggestions**

What changes or additional features would you suggest to improve the application? Please prioritize the improvements that would have had the biggest impact on your experience.

4.7 User Feedback Collection Template

This section reports the qualitative feedback collected from the testing user. The participant answered the following open-ended questions.

4.7.1 Questionnaire Responses

1. Overall experience

After using the application for approximately 48 hours, how would you describe your overall experience?

"The app worked really well and was easy to use. I remained surprised by how it could clearly show on which products I spent more money on. The most problematic thing was that I couldn't put it in a widget on the home page."

2. Usability and navigation

How easy was it to understand and navigate the application's main features?

"The app was easy to understand and use."

3. Functionality and reliability

Did the application behave as you expected during your usage period?

The app behaved perfectly and didn't show any signs of bugs or sorts of things.

4. Usefulness and value

In your opinion, how useful is the application for its intended purpose?

"The app could be useful for any customer. It helped me have a clear understanding on how I spent my money."

5. Improvements and suggestions

What changes or additional features would you suggest to improve the application?

"It would be nice if it showed the feedbacks of the nearest supermarkets too."

5 | Future Works

List It is deployment ready. It correctly satisfies every function requirement while providing its services to the end users through a minimal and intuitive interface.

In order to improve the app's utility, there have been identified further possible app extensions. These improvements do not substitute in any way the app's current functionalities. They are proposed as a guideline for future development in order to keep the app updated and keep the user interested as newer versions are released.

5.1 Wearable Device

As smart watches are becoming more popular, more and more people use them for every day activities. Some may include, checking notifications and managing phone calls.

Given the portability of such devices, a possible future improvement of List It may consist in a portable version so to let the user have every list on their wrists. The consequence would be a hands free experience, which is fundamental to avoid managing the smartphone during shopping.

5.2 Shared Lists

The app may be extended in order to allow users to have shared lists. Users could add and remove products from these lists independently. This may require the implementation of a shared link or a following status between users accordingly. The advantage would be for groups of friends or families to gather multiple preferences when preparing the groceries for collective events.

5.3 Supermarket APIs

In conclusion, supermarket companies may be given the possibility to also access the app in a user role-based manner. Supermarkets representative could be able to provide their supermarkets aisle configuration directly. In this way, users could either personalize supermarkets or directly access the available ones without having to worry about customization. Provided supermarkets may further be differentiated by their category disposition, location or company.